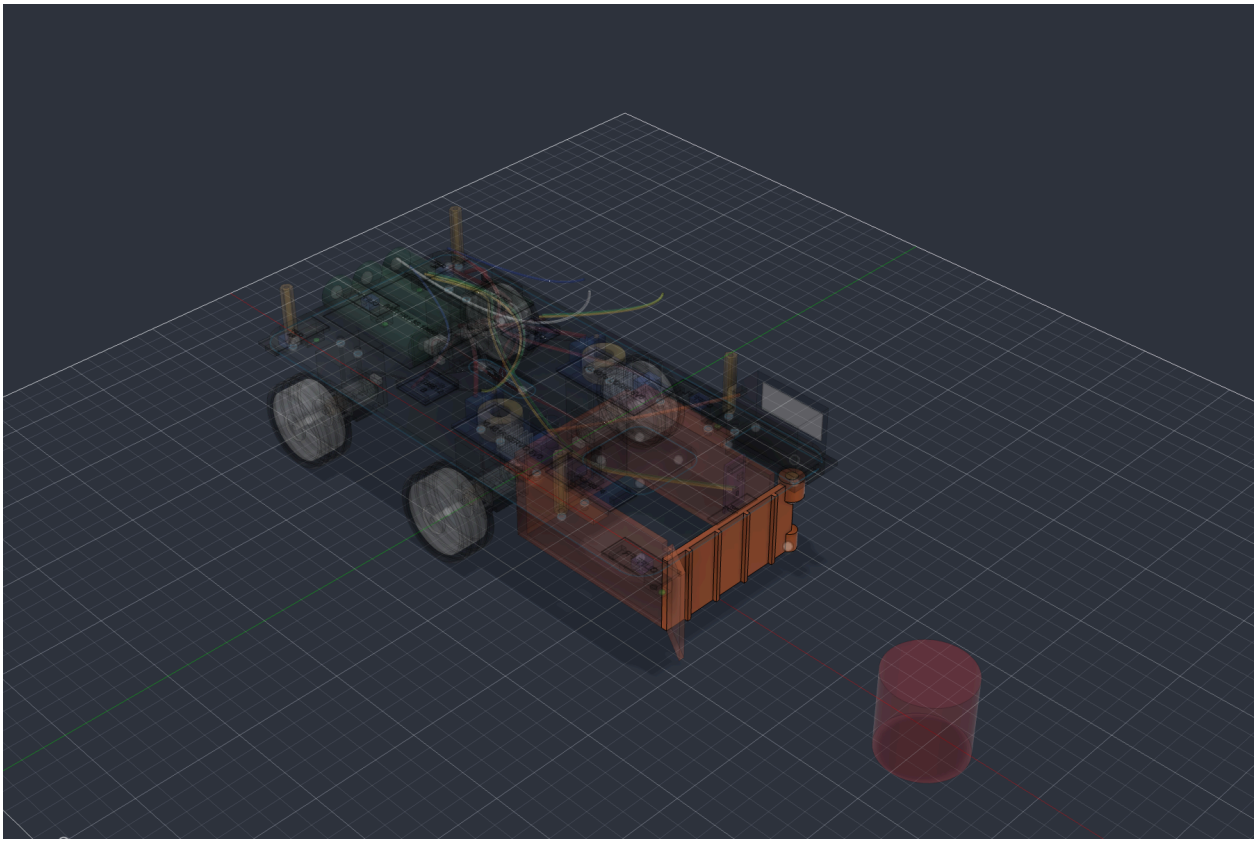
A.E.G.I.S. MK2

Deployment guide

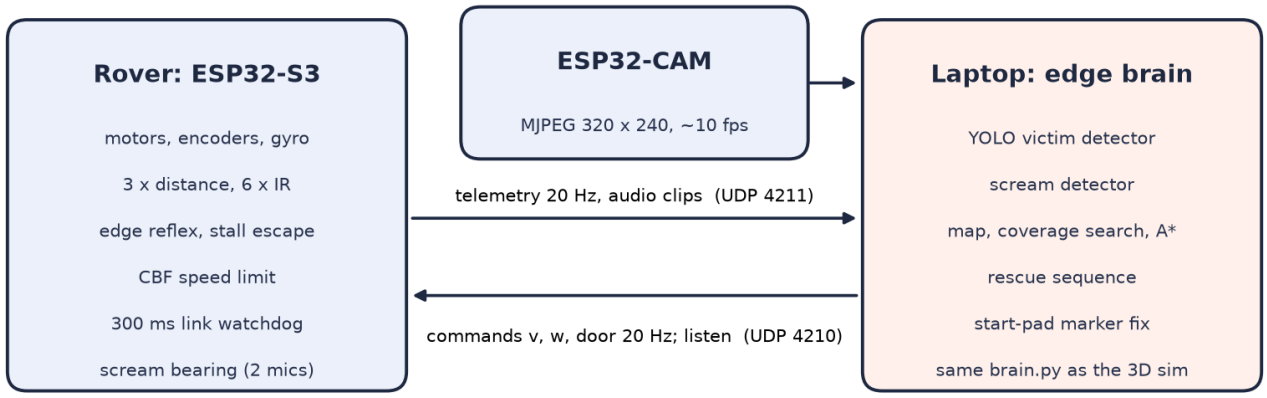
Flashing, network, bring-up and the first autonomous mission



Autonomous search-and-rescue rover
TL101 Tinkering Lab, IIT Ropar, AY 2026-27

Three programs make the system:

Runs on	Program	Source
ESP32-S3 (rover)	Motors, sensors, table-edge reflex, stall escape, CBF speed limit, link watchdog, scream bearing, Wi-Fi link	`firmware/aegis_s3/`
ESP32-CAM	Camera stream (MJPEG, 320 x 240, ~10 fps)	`firmware/aegis_cam/`
Laptop	Edge brain: detector, mapping, search, rescue, return home (the same `brain.py` the 3D simulation used)	`edge/aegis_edge.py`



What runs where, and how the three boards talk.

Safety never depends on Wi-Fi: if the laptop stops sending commands for 300 ms, the rover stops. The edge reflex and the front-distance limit run on the rover in every mode.

All commands below run from the `project` folder.

1. Network

Pick one 2.4 GHz network for the rover, the camera and the laptop. A **phone hotspot** is the easiest choice: it keeps the lab Wi-Fi out of the loop. The firmware default is SSID `AEGIS`, password `aegis1234`. Either name your hotspot that, or change it in step 2.

The laptop must allow incoming UDP on port 4211. On macOS, if the firewall asks, allow Python.

2. Flash the rover (ESP32-S3)

```
source ~/esp/idf_env.sh
```

```
cd team_share/firmware/aegis_s3 && idf.py menuconfig
```

Under **A.E.G.I.S. link**, set the Wi-Fi name and password, then save and quit. Then plug the S3 in by its **USB** port (not the UART one) and flash:

```
idf.py -p /dev/cu.usbmodem* flash monitor
```

On boot the log should show `ready, faults 0x00`, `Wi-Fi up, rover IP ...` and `UDP 4210 up`. A fault code means a missing sensor; the bring-up tool (step 5) says which one.

3. Flash the camera (ESP32-CAM)

Seat the ESP32-CAM on the ESP32-CAM-MB programmer board and plug it in:

```
cd team_share/firmware/aegis_cam && idf.py menuconfig
```

Set the same Wi-Fi under **A.E.G.I.S. camera link**, then:

```
idf.py -p /dev/cu.usbserial* flash monitor
```

The log prints `stream at http://<ip>/stream`. Open that address in a browser to check the picture. If it is upside down, set `CAM_FLIP` in `main/cam_main.c` to 0 and flash again. Then move the ESP32-CAM to the rover; it runs from the 5 V logic rail.

4. Print the start pad

```
~/aegis_venv/bin/python team_share/edge/make_start_pad.py
```

Print `edge/AEGIS_start_pad.pdf` on A4 at **100 % / actual size** and check that the marker measures 100 mm. Tape it flat on the table. Measure its centre from the table corner you will use as the origin: x along the long edge, y along the short edge, in metres. Those are your `--start` x and y. The arrow direction is the heading (0 = along x).

5. Bring-up (rover on a stand first)

```
~/aegis_venv/bin/python team_share/edge/bringup.py
```

It finds the rover by itself and shows every sensor live. Work through the checks:

1. **Faults: none.** Every fault line names the part and the wire to check.
2. **Distance sensors:** move a hand in front of each one; the value should follow it.
3. **Edge sensors:** on a stand every sensor reads `EDGE`. Put the rover on the table: all should read `ok`. Slide each sensor over the edge by hand: only that one should change. If every sensor reads the opposite way, set `CLIFF_ACTIVE_HIGH` to 0 in `components/aegis_core/include/aegis_params.h`.
4. **Bay sensor:** put the token in the bay; it should read `TOKEN INSIDE`. If inverted, set `CAPTURE_ACTIVE_LOW` to 0.
5. **Door:** press `o`. Adjust `SERVO_US_DOOR_OPEN` and `SERVO_US_DOOR_CLOSED` until the door opens fully and closes without pushing on the wall.
6. **Wheels:** with the rover still on the stand, press `w`. All four wheels should turn forward. A wheel turning backwards needs its two motor wires swapped; a wheel that does not turn means a driver or STBY problem.
7. **Microphones:** press `l` and clap on one side. You should get bearings with the sign matching that side (+ = left).

8. **On the table:** drive slowly towards an edge. The rover must stop and back off by itself before a wheel leaves the table. **Do this before any autonomous run.**

Then record real screams for the detector: press `r` and scream, about ten times, from different distances. The clips go to `simulation3d/screams/`.

6. Run a mission

```
~/aegis_venv/bin/python team_share/edge/aegis_edge.py --demo 2 --table 1.8 1.2 --start 0.20 0.20 0
```

- `--demo 1`: hidden token, no sound. `--demo 2`: a phone at the token plays a scream.
- `--table`: the table's size in metres. `--start`: the pad centre and heading (step 4).

A video window opens. Park the rover on the pad facing the arrow, then press **g** to start. **space** pauses (the rover stops) and **q** quits. Every run writes a log to `edge/logs/`.

7. Make the detector and scream detector see the real world

The detector was trained on rendered images. Retrain it on photos of your token before the demo:

```
~/aegis_venv/bin/python team_share/edge/capture_frames.py
```

The file's header explains what to photograph, how to label the photos (makesense.ai, free, in the browser) and the retraining command. Optionally, run `calibrate_camera.py` with a printed chessboard and pass `--calib team_share/edge/camera_calib.npz` to `aegis_edge.py` for a more accurate start-pad fix.

Testing without hardware

`fake_rover.py` stands in for the rover and the camera: it speaks the firmware's exact protocol, driven by the 3D simulator. `test_end_to_end.py` runs complete real-time missions through `aegis_edge.py` against it:

```
~/aegis_venv/bin/python team_share/edge/test_end_to_end.py --demo 1 --seeds 3000 3001 3002
```

Protocol (for debugging)

Direction	Port	Message
laptop to rover	UDP 4210	`{"type":"cmd","v":0.12,"w":-0.4,"door":1}` at 20 Hz; `{"type":"listen","dur":0.8}`
rover to laptop	UDP 4211	`{"type":"telem",...}` at 20 Hz (broadcast at 2 Hz until a laptop connects)
rover to laptop	UDP 4211	`AUD1` audio chunks, then `{"type":"audio_meta",...}`
camera to laptop	UDP 4211	`{"type":"cam","url":"http://<ip>/stream"}` every 2 s
laptop from camera	HTTP 80	`/stream` (MJPEG), `/capture` (one JPEG)